

Assignment-16.4

Name: Arjun Manoj

Batch:44

H.No:2303A52134

Task 1: Database Schema Design for an Online Course Platform

Prompt:

Design a relational database schema for an online learning platform that manages instructors, courses, and students. The schema should include proper primary keys, foreign key relationships, and follow normalization principles to ensure data integrity and avoid redundancy.

Code:

```
Assignment16.4 task1.sql
1 -- Create Guide Table (Instructors)
2 CREATE TABLE guide_info (
3     guide_id INT PRIMARY KEY,
4     guide_name VARCHAR(100),
5     domain VARCHAR(100),
6     region VARCHAR(50)
7 );
8
9 -- Create Program Table (Courses)
10 CREATE TABLE program_data (
11     program_id INT PRIMARY KEY,
12     program_title VARCHAR(100),
13     weeks INT,
14     guide_id INT,
15     FOREIGN KEY (guide_id) REFERENCES guide_info(guide_id)
16 );
17
18 -- Create Participant Table (Students)
19 CREATE TABLE participant_records (
20     participant_id INT PRIMARY KEY,
21     participant_name VARCHAR(100),
22     email_id VARCHAR(100),
23     program_id INT,
24     FOREIGN KEY (program_id) REFERENCES program_data(program_id)
25 );
26
27 -- INSERT DATA
28 INSERT INTO guide_info VALUES
29 (1, 'Adwait', 'AI', 'India'),
30 (2, 'Arjun', 'Cloud', 'UK');
31
32 INSERT INTO program_data VALUES
33 (201, 'Deep Learning', 12, 1),
34 (202, 'AWS Basics', 8, 2);
35
36 INSERT INTO participant_records VALUES
37 (3001, 'Sushma', 'sushma@gmail.com', 201),
38 (3002, 'Navya', 'navya@gmail.com', 202);
39
40 -- OUTPUT
41 SELECT * FROM participant_records;
```

SQL < 1 / 1 > 1 - 0 of 0

SQL < 1 / 1 > 1 - 0 of 0

SQL < 1 / 1 > 1 - 0 of 0

SQL < 1 / 1 > 1 - 0 of 0

SQL < 1 / 1 > 1 - 0 of 0

SQL < 1 / 1 > 1 - 2 of 2

participant_id	participant_name	email_id	program_id
3001	Sushma	sushma@gmail.com	201
3002	Navya	navya@gmail.com	202

Observation:

- Separate tables reduce data duplication
- Primary keys uniquely identify each record
- Foreign keys connect related tables correctly
- Structure follows basic normalization rules
- Ensures accurate and consistent data

Task 2: Populate the Database with Sample Records

Prompt:

Insert sample data into the database tables to test functionality. Ensure valid relationships between instructors, courses, and students, and verify correctness using SELECT queries.

Code:

```
Assignment16.4 task2.sql
1  -- =====
2  -- CREATE TABLES
3  -- =====
4
5  -- Coach Table (Instructors)
6  CREATE TABLE coach_master (
7      coach_id INT PRIMARY KEY,
8      coach_name VARCHAR(100),
9      skill_area VARCHAR(100),
10     country_name VARCHAR(50)
11 );
12
13 -- Course Table
14 CREATE TABLE course_hub (
15     course_id INT PRIMARY KEY,
16     course_title VARCHAR(100),
17     duration_weeks INT,
18     coach_id INT,
19     FOREIGN KEY (coach_id) REFERENCES coach_master(coach_id)
20 );
21
22 -- Student Table
23 CREATE TABLE student_hub (
24     student_id INT PRIMARY KEY,
25     student_name VARCHAR(100),
26     email VARCHAR(100),
27     course_id INT,
28     FOREIGN KEY (course_id) REFERENCES course_hub(course_id)
29 );
30
31 -- =====
32 -- INSERT DATA
33 -- =====
34
35 -- Instructors
36 INSERT INTO coach_master VALUES
37 (1, 'Kiran', 'AI', 'India'),
38 (2, 'Sneha', 'Cloud', 'UK'),
39 (3, 'Rahul', 'Data Science', 'USA');
40
41 -- Courses
42 INSERT INTO course_hub VALUES
43 (101, 'Deep Learning', 12, 1),
44 (102, 'AWS Basics', 8, 2),
45 (103, 'Data Analytics', 10, 3),
46 (104, 'Neural Networks', 14, 1);
47
48 -- Students
49 INSERT INTO student_hub VALUES
50 (201, 'Anjali', 'anjali@gmail.com', 101),
51 (202, 'Vikram', 'vikram@gmail.com', 102),
52 (203, 'Priya', 'priya@gmail.com', 103);
53
54 -- =====
55 -- OUTPUT (VERIFY)
56 -- =====
57
58 SELECT * FROM coach_master;
59 SELECT * FROM course_hub;
60 SELECT * FROM student_hub;
```

SQL > 1 / 1 > 1 - 0 of 0

SQL > 1 / 1 > 1 - 0 of 0

SQL > 1 / 1 > 1 - 0 of 0

SQL > 1 / 1 > 1 - 0 of 0

SQL > 1 / 1 > 1 - 0 of 0

SQL > 1 / 1 > 1 - 0 of 0

SQL > 1 / 1 > 1 - 3 of 3

coach_id	coach_name	skill_area	country_name
1	Kiran	AI	India
2	Sneha	Cloud	UK
3	Rahul	Data Science	USA

SQL > 1 / 1 > 1 - 4 of 4

course_id	course_title	duration_weeks	coach_id
101	Deep Learning	12	1
102	AWS Basics	8	2
103	Data Analytics	10	3
104	Neural Networks	14	1

SQL > 1 / 1 > 1 - 3 of 3

student_id	student_name	email	course_id
201	Anjali	anjali@gmail.com	101
202	Vikram	vikram@gmail.com	102
203	Priya	priya@gmail.com	103

SQL > 1 / 1 > 1 - 0 of 0

SQL > 1 / 1 > 1 - 0 of 0

SQL > 1 / 1 > 1 - 0 of 0

SQL > 1 / 1 > 1 - 0 of 0

SQL > 1 / 1 > 1 - 0 of 0

SQL > 1 / 1 > 1 - 0 of 0

SQL > 1 / 1 > 1 - 3 of 3

coach_id	coach_name	skill_area	country_name
1	Kiran	AI	India
2	Sneha	Cloud	UK
3	Rahul	Data Science	USA

SQL > 1 / 1 > 1 - 4 of 4

course_id	course_title	duration_weeks	coach_id
101	Deep Learning	12	1
102	AWS Basics	8	2
103	Data Analytics	10	3
104	Neural Networks	14	1

SQL > 1 / 1 > 1 - 3 of 3

student_id	student_name	email	course_id
201	Anjali	anjali@gmail.com	101
202	Vikram	vikram@gmail.com	102
203	Priya	priya@gmail.com	103

Observation:

- Sample data inserted successfully without errors

- Foreign key relationships are maintained correctly
- `SELECT` queries display accurate records
- Data is consistent and properly linked

Task 3: Implement Core Database Operations (CRUD)

Prompt:

Perform `CRUD` (Create, Read, Update, Delete) operations on the database to manage course and student data. Retrieve records, update student details, and delete course data while ensuring correctness through verification queries.

Code:

```

1  -- Assignment16.4 task3.sql
2  -- CREATE TABLES
3  -- =====
4
5  -- Trainer Table
6  CREATE TABLE trainer_core (
7      trainer_id INT PRIMARY KEY,
8      trainer_name VARCHAR(100),
9      expertise_area VARCHAR(100),
10     country VARCHAR(50)
11 );
12
13 -- Program Table
14 CREATE TABLE training_programs (
15     program_id INT PRIMARY KEY,
16     program_name VARCHAR(100),
17     duration INT,
18     trainer_id INT,
19     FOREIGN KEY (trainer_id) REFERENCES trainer_core(trainer_id)
20 );
21
22 -- Participant Table
23 CREATE TABLE participants (
24     participant_id INT PRIMARY KEY,
25     participant_name VARCHAR(100),
26     email VARCHAR(100),
27     program_id INT,
28     FOREIGN KEY (program_id) REFERENCES training_programs(program_id)
29 );
30
31 -- =====
32 -- INSERT DATA
33 -- =====
34
35 INSERT INTO trainer_core VALUES
36 (1, 'Amit', 'AI', 'India'),
37 (2, 'Neha', 'Cloud', 'UK'),
38 (3, 'Rohan', 'Data Science', 'USA');
39
40 INSERT INTO training_programs VALUES
41 (501, 'Deep Learning', 12, 1),
42 (502, 'AWS Basics', 8, 2),
43 (503, 'Data Analytics', 10, 3),
44 (504, 'Neural Networks', 14, 1);
45

```

program_id	program_name	duration	trainer_id
501	Deep Learning	12	1
502	AWS Basics	8	2
503	Data Analytics	10	3
504	Neural Networks	14	1

participant_id	participant_name	email	program_id
701	Meera	meera@gmail.com	501

program_id	program_name	duration	trainer_id
504	Neural Networks	14	1

trainer_id	trainer_name	expertise_area	country
1	Amit	AI	India
2	Neha	Cloud	UK

```

47 (701, 'Meera', 'meera@gmail.com', 501),
48 (702, 'Karthik', 'karthik@gmail.com', 502),
49 (703, 'Divya', 'divya@gmail.com', 503);
50
51 -- READ (RETRIEVE)
52 -- =====
53
54 SELECT * FROM training_programs;
55
56 -- UPDATE OPERATION
57 -- =====
58
59 -- BEFORE UPDATE
60 SELECT * FROM participants WHERE participant_id = 701;
61
62 -- UPDATE EMAIL
63 UPDATE participants
64 SET email = 'meera_new@gmail.com'
65 WHERE participant_id = 701;
66
67 -- AFTER UPDATE
68 SELECT * FROM participants WHERE participant_id = 701;
69
70 -- DELETE OPERATION
71 -- =====
72
73 -- BEFORE DELETE
74 SELECT * FROM training_programs WHERE program_id = 504;
75
76 -- DELETE
77 DELETE FROM training_programs
78 WHERE program_id = 504;
79
80 -- AFTER DELETE
81 SELECT * FROM training_programs;
82
83 -- FINAL VERIFY
84 -- =====
85
86 SELECT * FROM trainer_core;
87
88 SELECT * FROM training_programs;
89
90 SELECT * FROM participants;
91

```

participant_id	participant_name	email	program_id
701	Meera	meera_new@gmail.com	501

program_id	program_name	duration	trainer_id
504	Neural Networks	14	1

program_id	program_name	duration	trainer_id
501	Deep Learning	12	1
502	AWS Basics	8	2
503	Data Analytics	10	3

trainer_id	trainer_name	expertise_area	country
1	Amit	AI	India
2	Neha	Cloud	UK
3	Rohan	Data Science	USA

program_id	program_name	duration	trainer_id
501	Deep Learning	12	1
502	AWS Basics	8	2
503	Data Analytics	10	3

participant_id	participant_name	email	program_id
701	Meera	meera_new@gmail.com	501
702	Karthik	karthik@gmail.com	502
703	Divya	divya@gmail.com	503

Observation:

- Data retrieved correctly using SELECT
- Participant details updated successfully
- Deleted record is no longer visible

- Before and after queries verify changes
- Data consistency maintained

Task 4: Reporting Using Aggregate Queries

Prompt:

Generate summary reports using aggregate SQL queries to analyze student enrollments. Use `GROUP BY` and `HAVING` clauses to count enrollments per course and filter courses based on minimum enrollment criteria.

Code:

The screenshot displays a SQL IDE interface with a script editor on the left and a query results pane on the right. The script defines three tables: `mentor_list`, `course_registry`, and `student_registry`, and inserts data into them. The query results pane shows the output of a query that counts the total number of students for each course.

```

1 -- =====
2 -- CREATE TABLES
3 -- =====
4
5 -- Instructor Table
6 CREATE TABLE mentor_list (
7   mentor_id INT PRIMARY KEY,
8   mentor_name VARCHAR(100),
9   field VARCHAR(100),
10  country VARCHAR(50)
11 );
12
13 -- Course Table
14 CREATE TABLE course_registry (
15   course_id INT PRIMARY KEY,
16   course_name VARCHAR(100),
17   mentor_id INT,
18   FOREIGN KEY (mentor_id) REFERENCES mentor_list(mentor_id)
19 );
20
21 -- Student Table
22 CREATE TABLE student_registry (
23   student_id INT PRIMARY KEY,
24   student_name VARCHAR(100),
25   course_id INT,
26   FOREIGN KEY (course_id) REFERENCES course_registry(course_id)
27 );
28
29 -- =====
30 -- INSERT DATA
31 -- =====
32
33 INSERT INTO mentor_list VALUES
34 (1, 'Dev', 'AI', 'India'),
35 (2, 'Isha', 'Cloud', 'UK'),
36 (3, 'Varun', 'Data Science', 'USA');
37
38 INSERT INTO course_registry VALUES
39 (801, 'Machine Learning', 1),
40 (802, 'Cloud Computing', 2),
41 (803, 'Big Data', 3);
42
43 INSERT INTO student_registry VALUES
44 (1, 'Asha', 801),
45 (2, 'Ravi', 801),
  
```

The query results pane shows the output of a query that counts the total number of students for each course. The results are as follows:

course_id	total_students
801	3
802	1
803	1

```
Assignment16.4 task4.sql
33 INSERT INTO mentor_list VALUES
34 (1, 'Dev', 'AI', 'India'),
35 (2, 'Isha', 'Cloud', 'UK'),
36 (3, 'Varun', 'Data Science', 'USA');
37
38 INSERT INTO course_registry VALUES
39 (801, 'Machine Learning', 1),
40 (802, 'Cloud Computing', 2),
41 (803, 'Big Data', 3);
42
43 INSERT INTO student_registry VALUES
44 (1, 'Asha', 801),
45 (2, 'Ravi', 801),
46 (3, 'Pooja', 802),
47 (4, 'Kunal', 801),
48 (5, 'Neeraj', 803);
49
50 -- =====
51 -- AGGREGATE QUERY 1
52 -- Count students per Course
53 -- =====
54
55 SELECT course_id, COUNT(student_id) AS total_students
56 FROM student_registry
57 GROUP BY course_id;
58
59 -- =====
60 -- AGGREGATE QUERY 2
61 -- Courses with more than 2 students
62 -- =====
63
64 SELECT course_id, COUNT(student_id) AS total_students
65 FROM student_registry
66 GROUP BY course_id
67 HAVING COUNT(student_id) > 2;
```

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 3 of 3

course_id	total_students
801	3
802	1
803	1

SQL ▾ < 1 / 1 > 1 - 1 of 1

course_id	total_students
801	3

Observation:

- *GROUP BY* correctly groups students by course
- *COUNT()* gives accurate enrollment numbers
- *HAVING* filters courses based on condition
- Results are correct and meaningful
- Queries useful for reporting and analysis

Task 5: Query Optimization with AI Suggestions

Prompt:

Analyze and optimize a SQL query that retrieves courses taught by instructors from a specific country. Improve

performance using efficient joins and indexing while ensuring the output remains unchanged.

Code:

The image displays two screenshots of a SQL IDE interface, showing the transformation of a query for better performance.

Top Screenshot: Original (Unoptimized Query)

```
1  -- =====
2  -- CREATE TABLES
3  -- =====
4
5  -- Instructor Table
6  CREATE TABLE faculty_team (
7      faculty_id INT PRIMARY KEY,
8      faculty_name VARCHAR(100),
9      country VARCHAR(50)
10 );
11
12 -- Course Table
13 CREATE TABLE course_offerings (
14     offering_id INT PRIMARY KEY,
15     offering_name VARCHAR(100),
16     faculty_id INT,
17     FOREIGN KEY (faculty_id) REFERENCES faculty_team(faculty_id)
18 );
19
20 -- =====
21 -- INSERT DATA
22 -- =====
23
24 INSERT INTO faculty_team VALUES
25 (1, 'Aditya', 'India'),
26 (2, 'Sara', 'USA'),
27 (3, 'Nikhil', 'India');
28
29 INSERT INTO course_offerings VALUES
30 (901, 'AI Fundamentals', 1),
31 (902, 'Data Mining', 2),
32 (903, 'Deep Learning', 3);
33
34 -- =====
35 -- ORIGINAL (UNOPTIMIZED QUERY)
36 -- =====
37
38 SELECT offering_name
39 FROM course_offerings
40 WHERE faculty_id IN (
41     SELECT faculty_id
42     FROM faculty_team
43     WHERE country = 'India'
44 );
```

The results pane shows the output of the unoptimized query, displaying a list of offering names: AI Fundamentals and Deep Learning.

Bottom Screenshot: Optimized Query (JOIN)

```
38 SELECT offering_name
39 FROM course_offerings co
40 WHERE faculty_id IN (
41     SELECT faculty_id
42     FROM faculty_team ft
43     WHERE ft.country = 'India'
44 );
45
46 -- =====
47 -- OPTIMIZED QUERY (JOIN)
48 -- =====
49
50 SELECT co.offering_name
51 FROM course_offerings co
52 INNER JOIN faculty_team ft
53 ON co.faculty_id = ft.faculty_id
54 WHERE ft.country = 'India';
55
56 -- =====
57 -- INDEXING (OPTIONAL)
58 -- =====
59
60 CREATE INDEX idx_country ON faculty_team(country);
61 CREATE INDEX idx_faculty ON course_offerings(faculty_id);
62
63 -- =====
64 -- VERIFY OUTPUT
65 -- =====
66
67 SELECT co.offering_name
68 FROM course_offerings co
69 INNER JOIN faculty_team ft
70 ON co.faculty_id = ft.faculty_id
71 WHERE ft.country = 'India';
```

The results pane shows the output of the optimized query, displaying a list of offering names: AI Fundamentals and Deep Learning.

Observation:

Original query uses subquery (IN) → slower for large data

Optimized query uses JOIN → faster and efficient

Indexing improves search and join performance

Both queries return same results

Optimized query is more readable and scalable